

Alex Yang

236-889-8459 | alexemail67@gmail.com | [linkedin.com/in/alex-yang-](https://www.linkedin.com/in/alex-yang-) | github.com/alexyangg | alexyang.vercel.app

EDUCATION

University of British Columbia

Bachelor of Science - Computer Science (Co-op); GPA: 85%

Expected May 2027

Vancouver, BC

TECHNICAL SKILLS

Languages: Python, Java, C++, SQL, TypeScript, JavaScript, Ruby, HTML, CSS

Frameworks & Libraries: Spring Boot, gRPC, Ruby on Rails, React, Node.js, Express.js, Next.js, JUnit, pytest

Tools & Technologies: Git, Kafka, PostgreSQL, MongoDB, Amazon Web Services (AWS), Docker, Linux, Redis, npm

TECHNICAL EXPERIENCE

Tesla

Software Engineer Intern

May. 2026 – Aug. 2026

Palo Alto, CA

- Incoming distributed systems engineer on Tesla's Energy team, working with **Scala**, **Akka**, and **InfluxDB** to build real-time cloud platforms for monitoring and controlling millions of energy devices

Shopify

Software Engineer Intern

Jan. 2026 – Apr. 2026

Toronto, ON

- Led the migration of Shopify's Privacy Portal from a standalone **Ruby on Rails** app to an edge-rendered **React** app on **Cloudflare Workers**, reducing origin coupling for **130M+** unique visitors monthly
- Implemented **14** privacy request flows using **TypeScript** forms with server-validated contracts and client-side Global Privacy Control (GPC) detection, ensuring reliable request processing across **21** locales
- Owned cross-repository backend integration by building **GraphQL** queries, mutations, and schema changes that enabled server-to-server communication, resolved locale mismatches, and reduced client-side **CORS/CSRF** risk
- Strengthened high-abuse request endpoints by integrating **hCaptcha** and rate-limiting rules to mitigate bot-driven spam, maintaining **99.9%** availability and protecting service reliability during traffic spikes

Safe Software

Software Developer Intern

Sept. 2025 – Dec. 2025

Surrey, BC

- Developed **Python** scripts to scan internal library dependencies using data from the NIST vulnerability database, implementing version parsing and rate limiting to identify security issues and achieve **95%** scan accuracy
- Optimized Python scanning scripts with caching, de-duplication, and multithreading, reducing runtime by **35x**
- Integrated **Jira** and **Slack APIs** into security pipelines to automatically create issue tickets and send alerts for new vulnerabilities detected in third-party components, eliminating **90%** of manual triage work
- Built a test suite from scratch with **130+** unit and integration tests using **pytest** to validate end-to-end functionality of an internal vulnerability scanning system, achieving **100%** code coverage and preventing regressions

Cadabble

Software Engineer Intern

Feb. 2025 – Mar. 2025

Remote

- Implemented **Next.js** backend API routes connected to a **Prisma**-managed **Neon Postgres** database, enabling real-time ingestion of service submissions from **20+** agents and unblocking downstream processing
- Integrated **Cloudinary** for image uploads, offloading media handling and improving API responsiveness by **1.5x**
- Optimized frontend components by implementing **Next.js** lazy loading, reducing visual layout shifts by **80%**

PROJECTS

Patient Management System | Java, Spring Boot, Kafka, PostgreSQL, Docker, AWS

[GitHub](#)

- Built a microservices system with patient, billing, and auth services, supporting **REST** and **gRPC** communication
- Deployed **PostgreSQL** in **Docker** containers on **AWS** with health checks and persistent storage

WasteNot (nwHacks2025) | React.js, Node.js, Express.js, MongoDB

[GitHub](#) | [Website](#)

- Spearheaded a **full-stack web app** that generates recipe ideas from user-provided ingredients
- Secured user data with **JWT** authentication and **bcrypt** for password hashing
- Worked with **2** teammates, ranking among the **top 10** of 81 submissions for the Telus Sponsor Prize at **nwHacks**

Komorebi (Discord Bot) | JavaScript, Node.js, Discord.js, MongoDB, Mongoose

[GitHub](#) | [Website](#)

- Built **34** commands, including moderation tools and a user economy system, increasing server engagement by **40%**
- Implemented real-time event handling for commands using **Discord.js**, reducing command latency by **25%**
- Scaled the bot to **30+** Discord servers with **13,000+** unique users, maintaining a **99.9%** uptime using **AWS EC2**